



# 現代通信技術導論

---

算术编码+字典编码

## 目录 | CONTENTS

1

算术编码→向极限挑战

2

字典模型算法

3

LZ77算法

4

LZW算法

## 算术编码→向极限挑战

---

假设

某个字符的出现概率为80%

该字符只需要  $-\log_2(0.8) = 0.322$  bit 编码

但Huffman编码

只能为Ta分配1 bit 0 或 1 的编码

整条信息的 80%

在压缩后都几乎相当于理想长度的 3 倍左右

# 算术编码→向极限挑战

---

怎样才能只输出 0.322 个 0 或 0.322 个 1 呢?

用剪刀把二进制位剪开?

➤ 算术编码

## 算术编码→向极限挑战

算术编码对整条信息(无论信息有多长), 其输出仅仅是一个数, 而且是一个介于 0 和 1 之间的二进制小数。

例如

算术编码

对某条信息的编码输出

为 1010001111

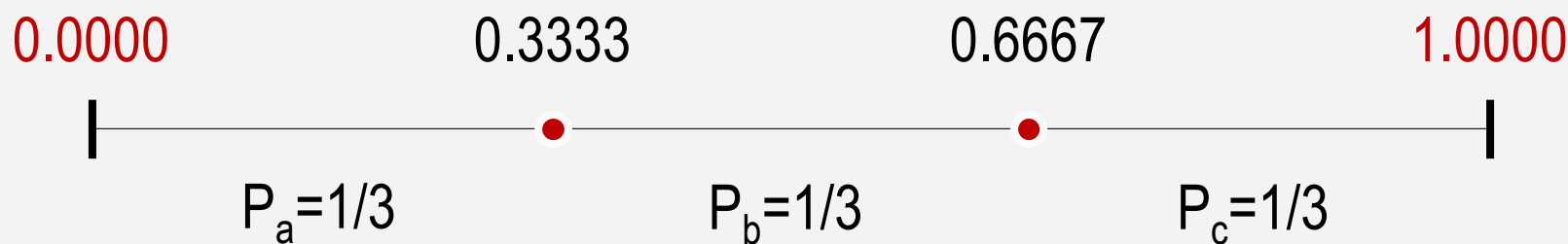
那么Ta表示小数 0.1010001111

也即十进制数 0.64

# 算术编码→向极限挑战

例：某条信息出现的字符仅  $a$   $b$   $c$  三种，要压缩信息为  $b c c b$

1. 压缩前，假设  $a$   $b$   $c$  在信息中的出现概率一无所知，先平均分配为  $1/3$ ;

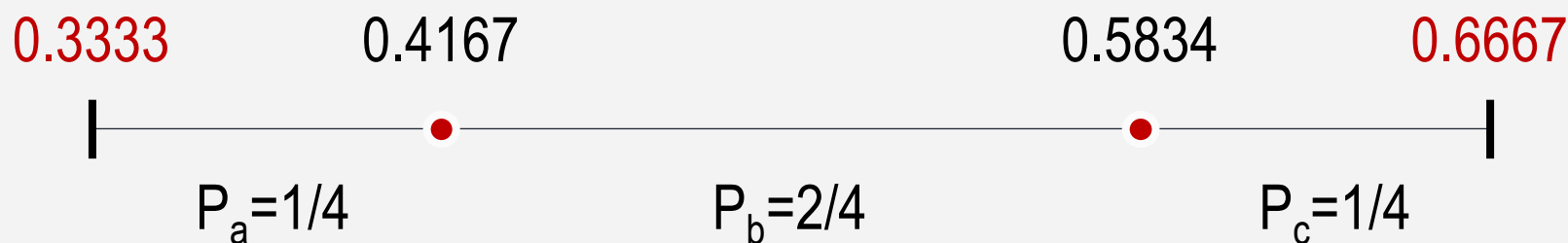


# 算术编码→向极限挑战

例：某条信息出现的字符仅 a b c 三种，要压缩信息为 **b c c b**

2. 取第一个字符b，找到b对应的区间(0.3333, 0.6667);

由于多了一个b，三字符的概率分布变成：  $P_a=1/4$ ,  $P_b=2/4$ ,  $P_c=1/4$ ;

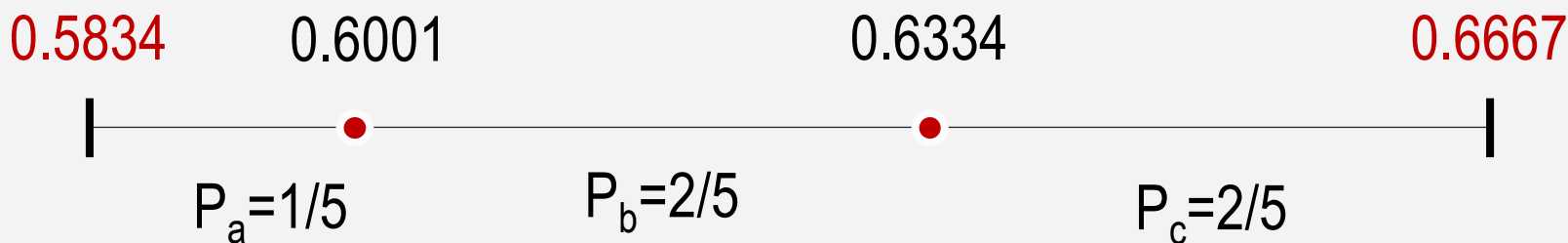


## 算术编码→向极限挑战

例：某条信息出现的字符仅  $a$   $b$   $c$  三种，要压缩信息为  $b c c b$

3. 接着压缩字符 $c$ ，取上一步 $c$ 的区间  $(0.5834, 0.6667)$ ;

新添了 $c$ 以后，三个字符的概率分布变成： $P_a=1/5$ ， $P_b=2/5$ ， $P_c=2/5$ ;

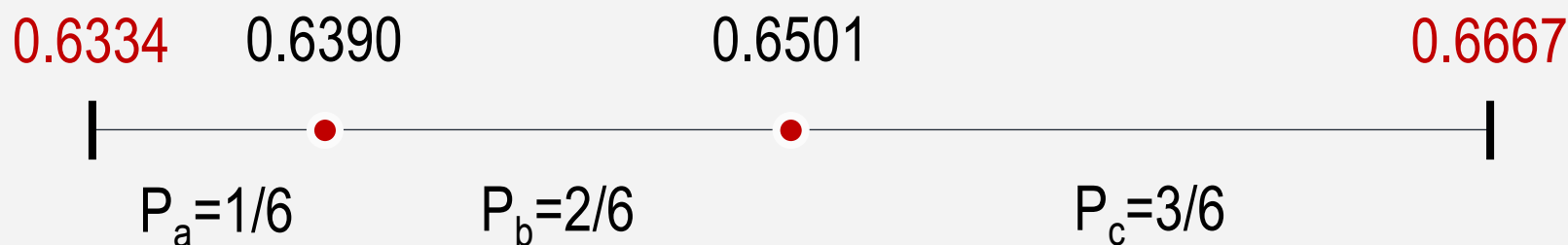




# 算术编码→向极限挑战

例：某条信息出现的字符仅  $a$   $b$   $c$  三种，要压缩信息为  $b c c b$

4. 又压缩 $c$ ，找到上一步中 $c$ 的区间  $(0.6334, 0.6667)$ ;  
再加一个 $c$ 后，三个字符的概率分布变成：  $P_a=1/6$ ,  $P_b=2/6$ ,  $P_c=3/6$ ;



# 算术编码→向极限挑战

---

例：某条信息出现的字符仅  $a$   $b$   $c$  三种，要压缩信息为  $b$   $c$   $c$   $b$

5. 下一个是字符 $b$ ，从上一步中得到 $b$ 的区间为 $(0.6390, 0.6501)$ ；  
因为是最后一个字符，不用进一步划分。



## 算术编码→向极限挑战

例：某条信息出现的字符仅 a b c 三种，要压缩信息为 **b c c b**

6. 在这个区间内选择一个容易变成二进制的数，如0.64，将Ta变成二进制  
0.1010001111，去掉前面没有意义的0和小数点，输出**1010001111**



# 目录 | CONTENTS

1

算术编码→向极限挑战

2

字典模型算法

3

LZ77算法

4

LZW算法

统计编码  
直到20世纪70年代末期  
仍占据着统治地位

“

一旦某项技术形成了惯例，人们就很难创造出在思路上与Ta大相径庭的哪怕是更简单更实用的技术。

”

致敬

两位在数据压缩领域做出了杰出贡献的以色列大神



Abraham Lempel



Jacob Ziv

正是他们打破了Huffman编码一统天下的格局，  
带来了既高效又简便的 “**字典模型**” 编码

# 字典模型算法

---

**字典模型**的思路相当简单

日常生活中就经常在使用这种压缩思想。

例：

NBA

MIT

LPF

# 字典模型算法

---

NBA

美国职业篮球联赛

MIT

麻省理工学院

LPF

低通滤波器



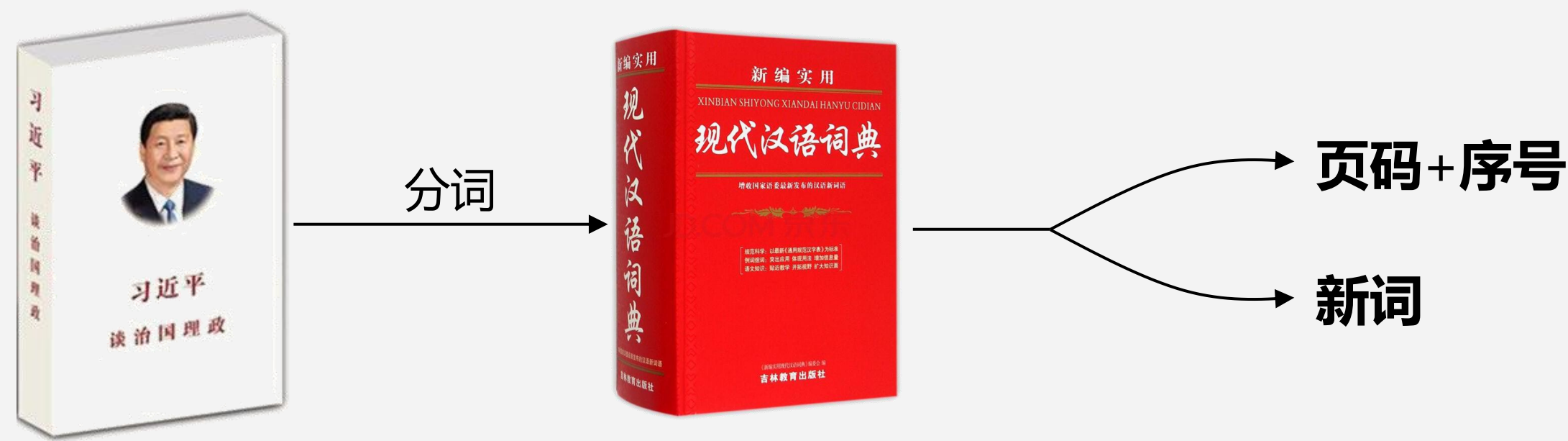
# 字典模型算法

---

→ NBA  
MIT →  
LPF  
...



# 静态字典模型



## 静态字典模型 - 缺点

---

1. 需要为每类不同的信息建立不同的字典，适应性不强。

2. 须维护信息量并不算小的字典，这也影响了最终的压缩效果。

# 自适应模型

---

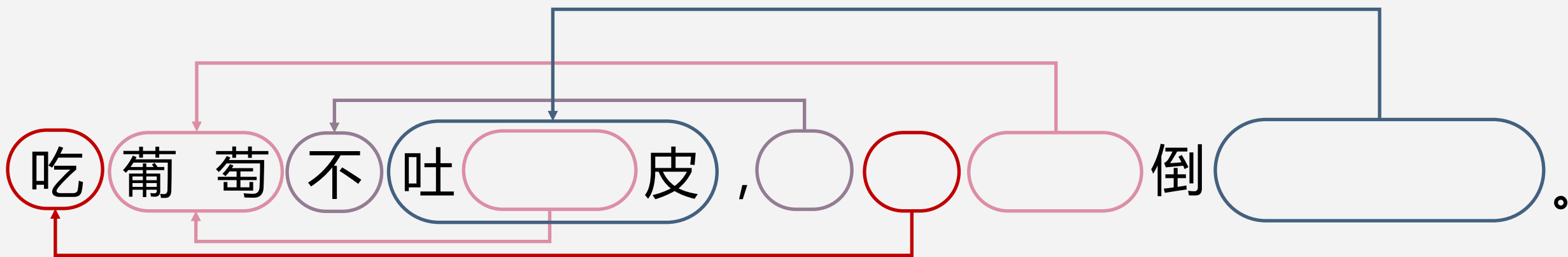
几乎所有通用的字典都使用了**自适应模型**。

也就是说，

将已经编码过的信息作为字典，  
如果要编码的字符串曾经出现过，  
就输出该字符串的出现位置及长度，  
否则输出新的字符串。

# 自适应模型

你能从下面这图中，还原出原始信息吗？



## 目录 | CONTENTS

1

算术编码→向极限挑战

2

字典模型算法

3

LZ77算法

4

LZW算法

# LZ77算法

---

## LZ77 算法

又可以称为 “滑动窗口压缩”

该算法将一个虚拟的，**可以跟随压缩进程滑动的窗口**

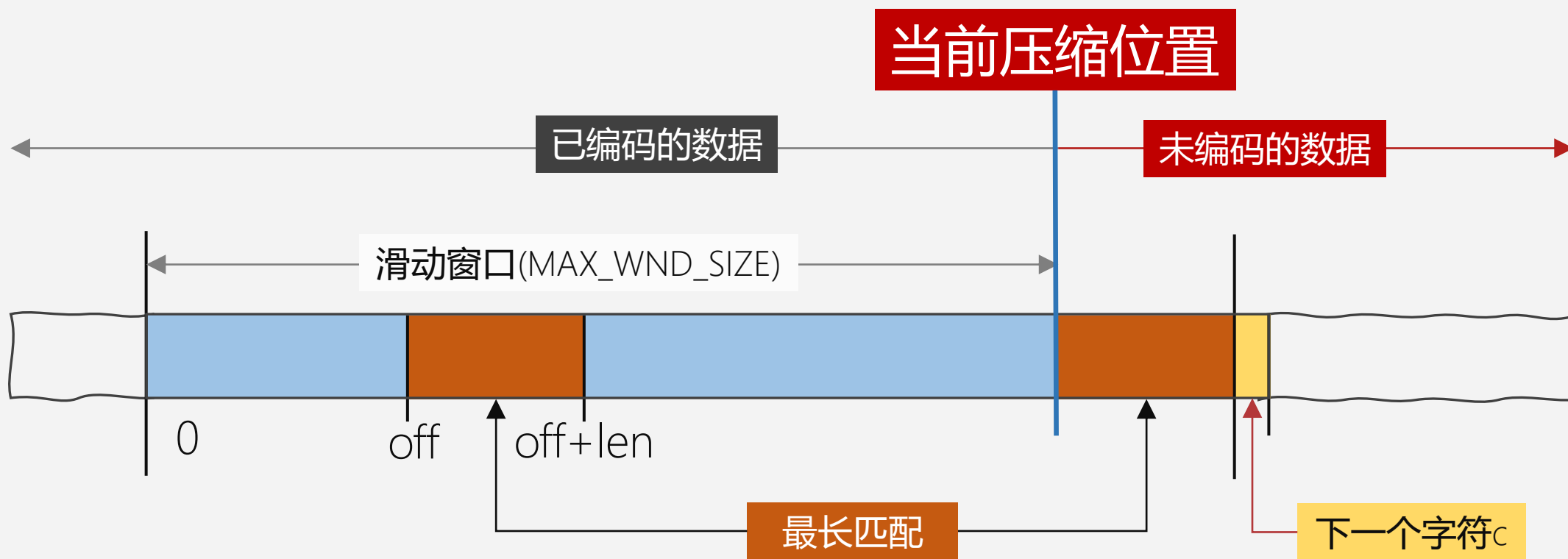
作为**术语字典**，

要压缩的字符串如果在该窗口中出现，

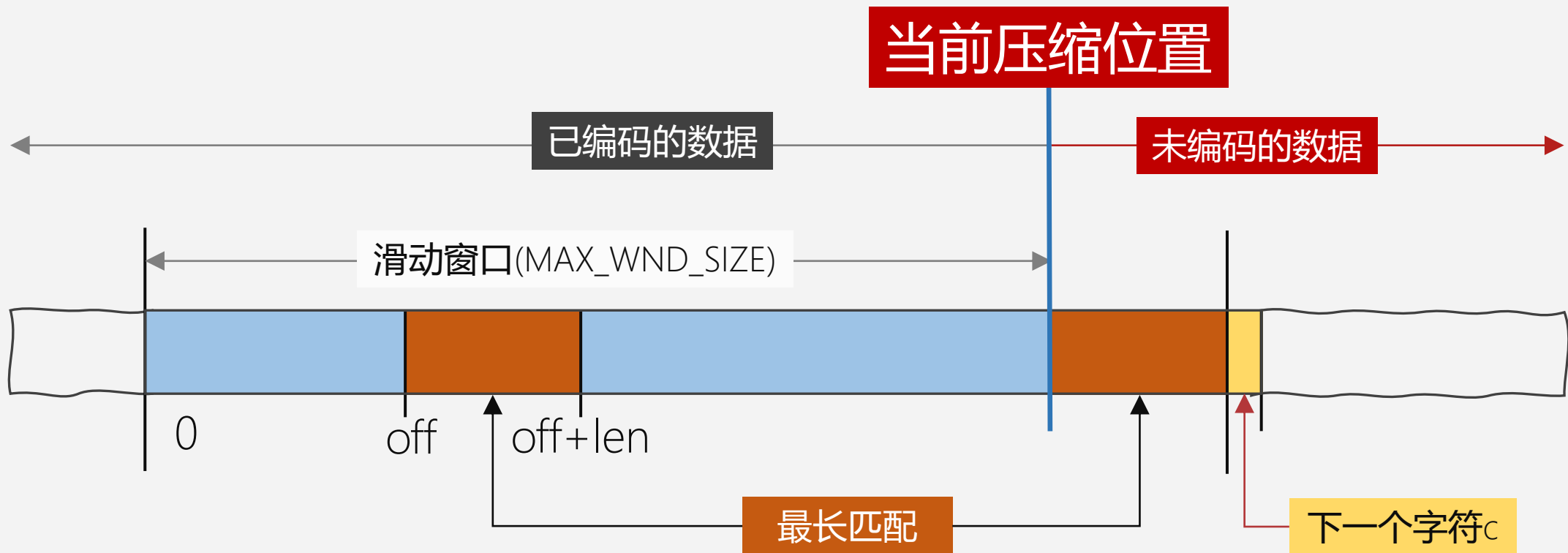
则输出其出现位置和长度。

# LZ77 算法

- LZ77 算法的基本流程







1. 从当前压缩位置开始，观察未编码数据，并在滑动窗口中找出最长的匹配字符串，如果找到，则进行步骤 2，否则进行步骤 3。
2. 输出三元符号组  $(off, len, c)$ 。其中  $off$  为窗口中匹配字符串相对窗口边界的偏移， $len$  为可匹配的长度， $c$  为下一个字符。然后将窗口向后滑动  $len + 1$  个字符，继续步骤 1。
3. 输出三元符号组  $(0, 0, c)$ 。其中  $c$  为下一个字符。然后将窗口向后滑动 1 个字符，继续步骤 1。

## LZ77算法 Q&A

---

Q1: 为什么用固定大小的窗口进行术语匹配?

A1: 之所以不是在所有已经编码的信息中匹配, 是因为匹配算法的时间消耗很大, 限制字典大小能保证算法效率;

Q2: 随着压缩进程, 滑动字典窗口始终仅包含刚编码过的信息, 这样合理吗?

A2: 合理, 因为对大多数信息而言, 要编码的字符串往往在最近的上下文中更容易找到匹配串。

# LZ77 算法

---

例：窗口大小为10个字符，刚编码过的10个字符为 “abcdbbccaa”，  
即将编码的11个字符为 “abaeaaabaee”。

1. 首先，和待编码字符匹配的最长串为ab(off=0,len=2), ab的下一个字符为a，输出三元组：(0,2,a)
2. 窗口向后滑动3个字符，窗口中的内容为：dbbccaaba
3. 下一个字符e在窗口中没有匹配，输出三元组：(0,0,e)
4. 窗口向后滑动1个字符，其中内容变为：bbccaabae
5. 接下来，要编码的aaabae在窗口中存在(off=4,len=6)，其后的字符为e，可以输出：(4,6,e)

# LZ77 算法

---

- 这样，将可以匹配的字符串都变成了指向窗口内的指针，并完成了对上述数据的压缩。
- 解压缩时，只要向压缩时那样维护好滑动的窗口，随着三元组的不断输入，在窗口中找到相应的匹配串，加上后继字符c输出（如果off和len都为0则只输出后继字符c）即可还原出原始数据。

## 目录 | CONTENTS

1

算术编码→向极限挑战

2

字典模型算法

3

LZ77算法

4

LZW算法

# LZW算法

---

LZW算法是LZ78的改进，  
其基本思路是在内存中维护一个动态的字典，  
输出的代码是该**字典的索引**

# LZW算法

---

例：待压缩的信息为 "DAD DADA DADDY DADO..."

|    |      |
|----|------|
| 索引 | 0    |
| 词条 | null |

1. 压缩串 "DAD..."，在内存词典中无法找到匹配串，  
则输出二元组(0, "D")

词典的索引

后续字符

# LZW算法

---

例：待压缩的信息为 "DAD DADA DADDY DADO..."

| 索引 | 0    | 1   |
|----|------|-----|
| 词条 | null | "D" |

将字符串 "D" 置入内存词典

2. 压缩串 "AD ..."，在内存词典中仍无法找到匹配串，  
则输出二元组(0, "A" )



# LZW算法

---

例：待压缩的信息为 "DAD DADA DADDY DADO..."

| 索引 | 0    | 1   | 2   |
|----|------|-----|-----|
| 词条 | null | "D" | "A" |

将字符串 "A" 置入内存词典

3. 压缩串 "D D..."，在内存词典中找到最大匹配串"D"，则输出二元组(1, "∪")

# LZW算法

---

例：待压缩的信息为 "DAD DADA DADDY DADO..."

| 索引 | 0    | 1   | 2   | 3    |
|----|------|-----|-----|------|
| 词条 | null | "D" | "A" | "D " |

将字符串 "D " 置入内存词典

4. 压缩串 "DAD..."，在内存词典中找到最大匹配串"D"，  
则输出二元组(1, "A" )

# LZW算法

---

例：待压缩的信息为 "DAD DADA DADDY DADO..."

| 索引 | 0    | 1   | 2   | 3    | 4    |
|----|------|-----|-----|------|------|
| 词条 | null | "D" | "A" | "D " | "DA" |

将字符串 "DA" 置入内存词典

...

# LZW算法

例：待压缩的信息为 "DAD DADA DADDY DADO..."

第九步后的内存词典

|    |       |       |      |      |        |
|----|-------|-------|------|------|--------|
| 索引 | 0     | 1     | 2    | 3    | 4      |
| 词条 | null  | "D"   | "A"  | "D " | "DA"   |
| 索引 | 5     | 6     | 7    | 8    | 9      |
| 词条 | "DA " | "DAD" | "DY" | " "  | "DADO" |

每一步的输出二元组

|    |          |          |          |          |          |
|----|----------|----------|----------|----------|----------|
| 步骤 | 1        | 2        | 3        | 4        | 5        |
| 输出 | (0, "D") | (0, "A") | (1, " ") | (1, "A") | (4, " ") |
| 步骤 | 6        | 7        | 8        | 9        |          |
| 输出 | (4, "D") | (1, "Y") | (0, " ") | (6, "O") |          |

# 课堂练习

- 利用LZW算法来压缩下面的信息

$$PQP \cup PQPQ \cup PQPY \dots$$

注： $\cup$  表示空格。

- 请把每一步的内存词典和输出的二元组填入下表。

|    |      |   |   |   |   |   |   |   |
|----|------|---|---|---|---|---|---|---|
| 索引 | 0    | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
| 词条 | Null |   |   |   |   |   |   |   |
| 输出 | 空    |   |   |   |   |   |   |   |